

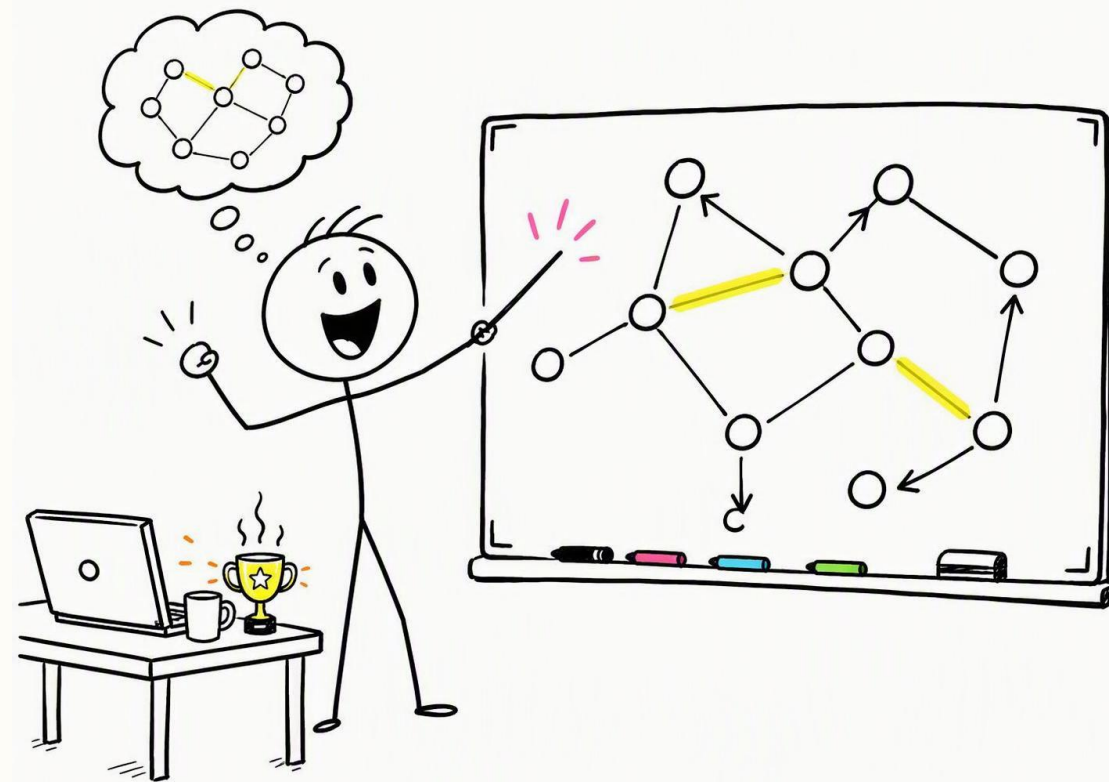
Teoria de Grafos para Programação Competitiva

Teoria · Algoritmos · Implementações em C++

Prof. Leovel Camilo Neto

Material de apoio para estudantes

Estruturas de Dados · DFS/BFS · Conectividade · Ordenação Topológica · MST · Caminhos Mínimos · Circuitos Eulerianos



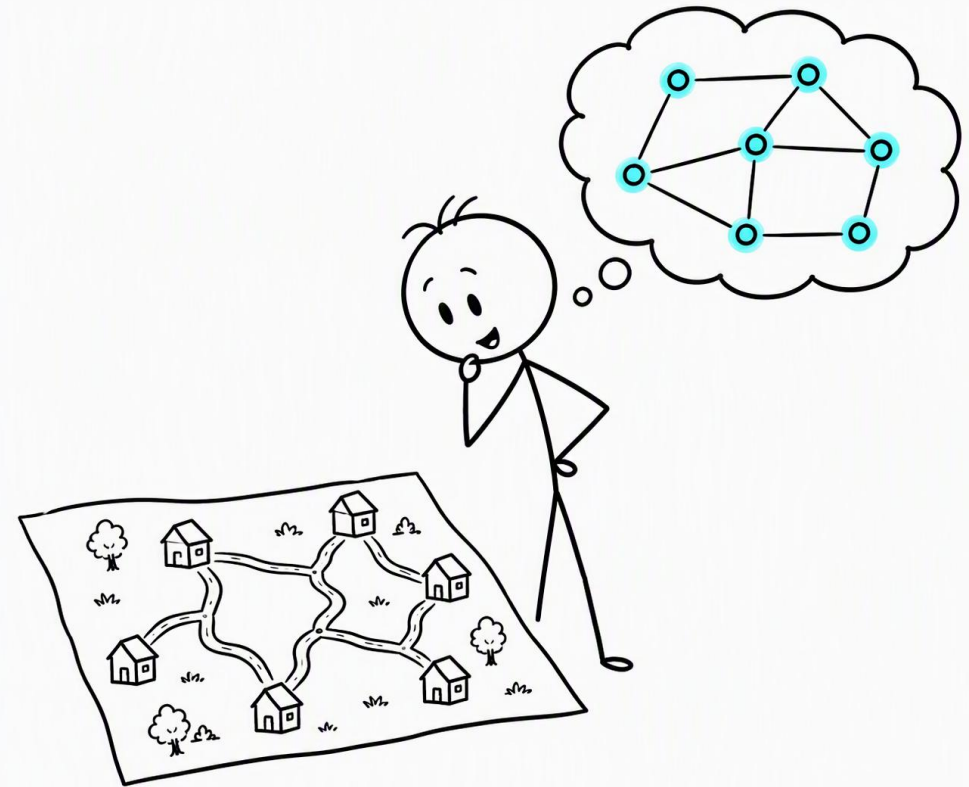
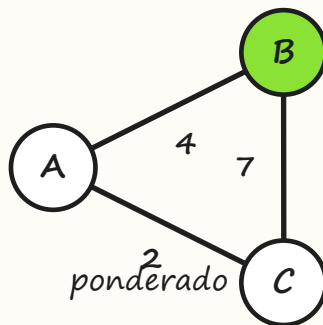
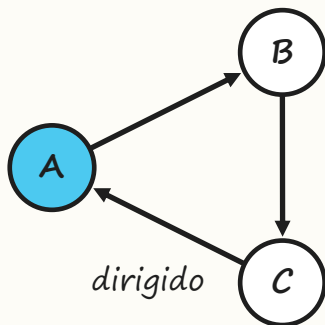
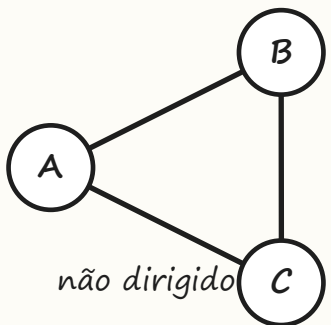
Afinal, o que é um grafo?

INTRODUÇÃO

$$G = (V, E)$$

$V = \text{vértices (nós)}$ · $E \subseteq V \times V = \text{arestas (arcos)}$

- ▶ Modelar o problema como grafo + escolher o algoritmo certo (com limites de tempo e memória) é o coração do IOI/ICPC.
- ▶ **Dirigido ou não?** as arestas têm sentido?
- ▶ **Ponderado?** existe $w : E \rightarrow \mathbb{R}$?
- ▶ **Estrutura especial?** árvore, DAG, bipartido, restrições de conectividade...



Mapa $\rightarrow$ pontos e linhas = grafo!

Roteiro da aventura

ÍNDICE

- 1 Representações e complexidade
- 2 DFS e BFS
- 3 Componentes conexos e Flood Fill
- 4 Ordenação topológica
- 5 Bipartição e detecção de ciclos
- 6 Pontos de articulação e pontes

- 7 Componentes fortemente conexas
- 8 Árvores de cobertura mínima
- 9 Caminhos mínimos (SSSP e APSP)
- 10 Grafos especiais e Euler
- 11 Quadro de decisão em prova

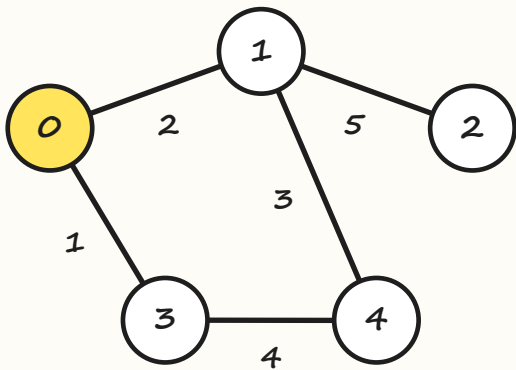
Dica nerd: cada tópico = 1 ferramenta na sua caixa de competição 🧰

Representar um grafo: 3 sabores

1 · ESTRUTURAS

A estrutura escolhida define a complexidade de TODAS as consultas e travessias seguintes.

Estrutura	Espaço	Adicionar aresta	Existe (u,v)?	Vizinhos de u	Cenário típico ICPC/IOI
Matriz de Adjacência (AM)	$O(V^2)$	$O(1)$	$O(1)$	$O(V)$	Grafos densos ($E \approx V^2$), $V \leq 1000$, operações de matriz
Lista de Adjacência (AL)	$O(V + E)$	$O(1)$ amortizado	$O(\text{grau}(u))$	$O(\text{grau}(u))$	Padrão ótimo para grafos esparsos ($V, E \leq 2 \times 10^5$)
Lista de Arestas (EL)	$O(E)$	$O(1)$	$O(E)$	$O(E)$	Kruskal e rotinas de ordenação global de arestas



grafo-exemplo usado nos próximos slides ($V = 5$)

Poucas arestas? → Lista de Adjacência

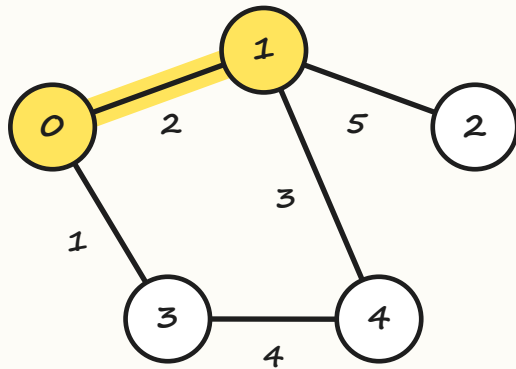
V pequeno e muitas consultas (u,v)? → Matriz

Vai ordenar arestas? → Lista de Arestas

Matriz de Adjacência

1 · ESTRUTURAS

- ▶ Tabela $|V| \times |V|$: $AM[u][v]$ guarda a existência ou o peso da aresta.
- ▶ **Só com $|V| \leq 1000$:** acima disso, estoura a memória do juiz.
- ▶ Inicialização: ∞ onde não há aresta e 0 na diagonal principal.



	0	1	2	3	4
0	0	2	∞	1	∞
1	2	0	5	∞	3
2	∞	5	0	∞	∞
3	1	∞	∞	0	4
4	∞	3	∞	4	0

simétrica: $AM[0][1] = AM[1][0] = 2$

Consulta (u,v) em $O(1)$... mas paga $O(V^2)$ de memória 🚧

C++ | Matriz de Adjacência

```
#include <vector>

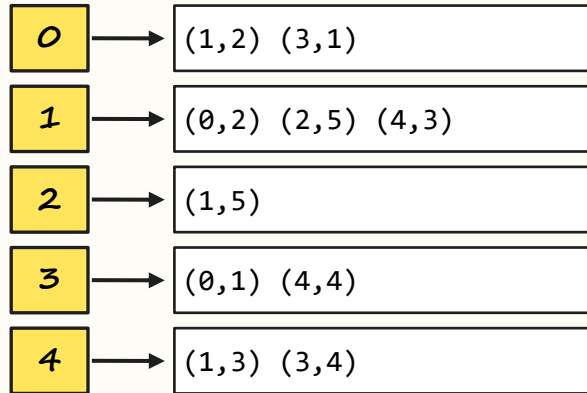
const int INF = 1e9;
int V = 5;
std::vector<std::vector<int>> AM(V, std::vector<int>(V, INF));

void adicionarArestaAM(int u, int v, int peso, bool dirigido = false) {
    AM[u][v] = peso;
    if (!dirigido) {
        AM[v][u] = peso;
    }
}
```

Lista de Adjacência e Lista de Arestas

1 · ESTRUTURAS

Lista de Adjacência: cada u guarda pares $(v, w) \rightarrow O(V + E)$



Itera só sobre vizinhos reais!

Lista de Arestas: um vetor de tuplos (peso, origem, destino)

w	u	v
1	0	3
2	0	1
3	1	4
4	3	4
5	1	2

← já ordenada por peso: pronta para o Kruskal!

Não responde rápido "quem são os vizinhos de u ?"

C++ | Lista de Adjacência

```
using ii = std::pair<int, int>;
using vii = std::vector<ii>;
int V = 5;
std::vector<vii> AL(V);

void adicionarArestaAL(int u, int v, int w, bool dirigido = false) {
    AL[u].push_back({v, w});
    if (!dirigido) {
        AL[v].push_back({u, w});
    }
}
```

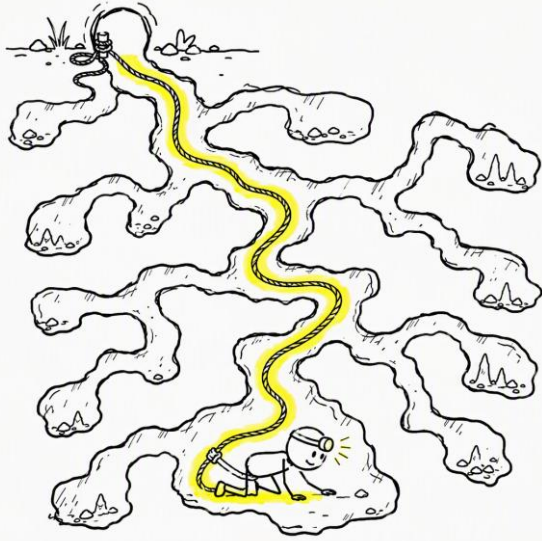
C++ | Lista de Arestas

```
#include <vector>
#include <tuple>

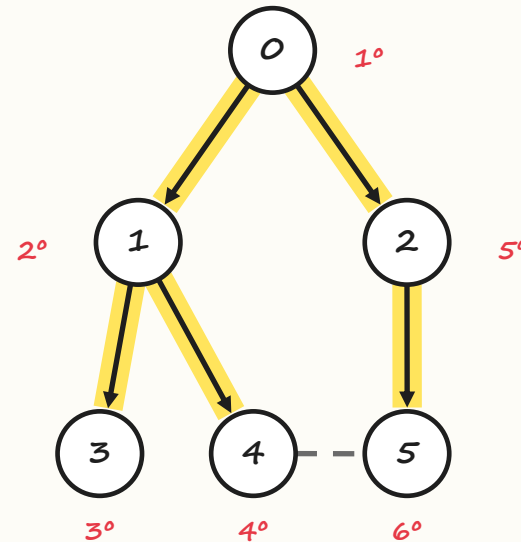
using iii = std::tuple<int, int, int>;
std::vector<iii> EL;

void adicionarArestaEL(int u, int v, int w) {
    EL.emplace_back(w, u, v);
}
```

DFS: mergulhar fundo, depois voltar



- ▶ Explora um ramo até o vértice mais profundo e só então retrocede (recursão).
- ▶ Usa a pilha de chamadas do sistema como rasto.
- ▶ Vetor `dfs_num` evita ciclos infinitos.
- ▶ **Tempo:** $O(V + E)$ com lista de adjacência.



Vai até ao fundo... e volta pela corda (backtrack)!

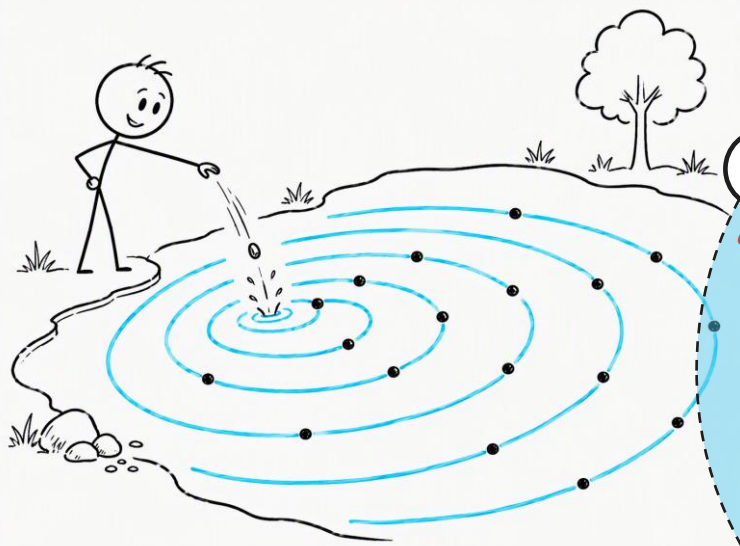
C++ | Pesquisa em Profundidade (DFS)

```
using ii = std::pair<int, int>;
using vii = std::vector<ii>;
using vi = std::vector<int>;

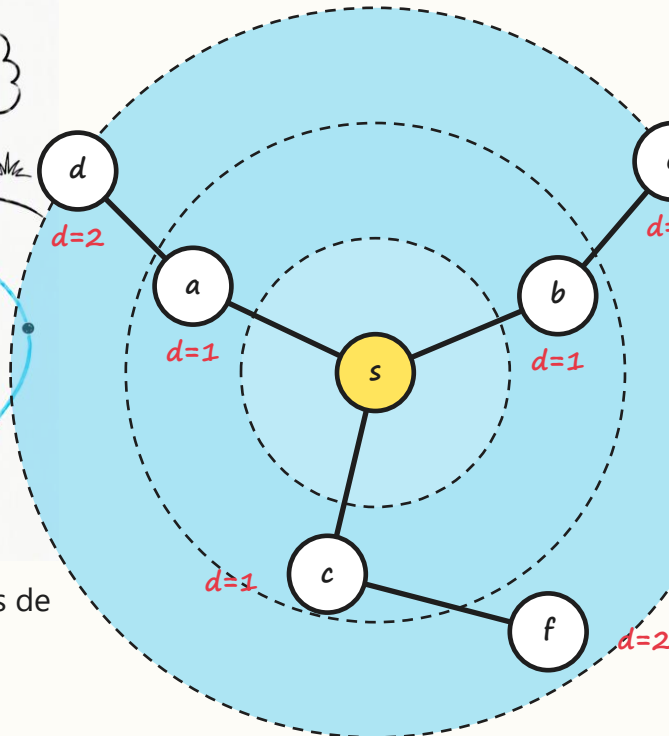
enum { NAO_VISITADO = -1, VISITADO = 1 };
std::vector<vii> AL;
vi dfs_num;

void dfs(int u) {
    dfs_num[u] = VISITADO;
    for (const auto &[v, w] : AL[u]) {
        if (dfs_num[v] == NAO_VISITADO) {
            dfs(v);
        }
    }
}
```

BFS: ondas concêntricas



- ▶ Visita todos a k arestas da origem antes dos de distância $k + 1$.
- ▶ Fila FIFO (`std::queue`).
- ▶ **Bônus:** caminho mínimo em n° de arestas em grafos não ponderados, $O(V + E)$.



Camada $0 \rightarrow 1 \rightarrow 2 \dots$ como ondas num lago

C++ | Pesquisa em Largura (BFS)

```
const int INF = 1e9;
std::vector<vii> AL;
vi dist;

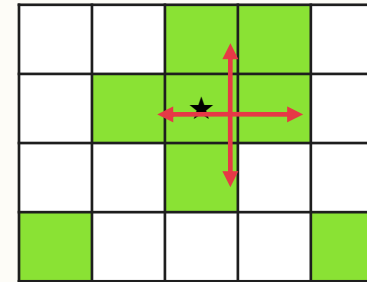
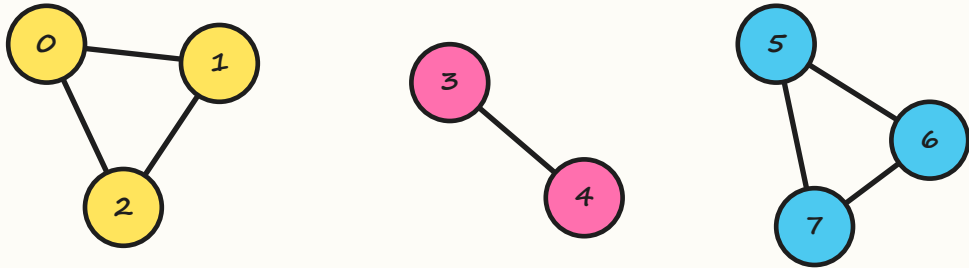
void bfs(int s, int V) {
    dist.assign(V, INF);
    std::queue<int> q;
    dist[s] = 0;
    q.push(s);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (const auto &[v, w] : AL[u]) {
            if (dist[v] == INF) {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
}
```

Componentes conexas e Flood Fill

3 · CONECTIVIDADE

Componente conexa = subgrafo conexo maximal. Percorre $u = 0 \dots V-1$: nó não visitado $\rightarrow$ nova DFS/BFS e $++$ contagem.

Grelha $R \times C$ = grafo implícito: cada célula é um vértice com 4 (ou 8) vizinhos. Sem lista de adjacência! $O(R \cdot C)$



Começa na ★ e "pinta" a mancha:
tamanho = 5

C++ | Contagem de Componentes Conexas

```
int contarComponentesConexas(int V) {
    dfs_num.assign(V, NAO_VISITADO);
    int totalCC = 0;
    for (int u = 0; u < V; ++u) {
        if (dfs_num[u] == NAO_VISITADO) {
            ++totalCC;
            dfs(u);
        }
    }
    return totalCC;
}
```

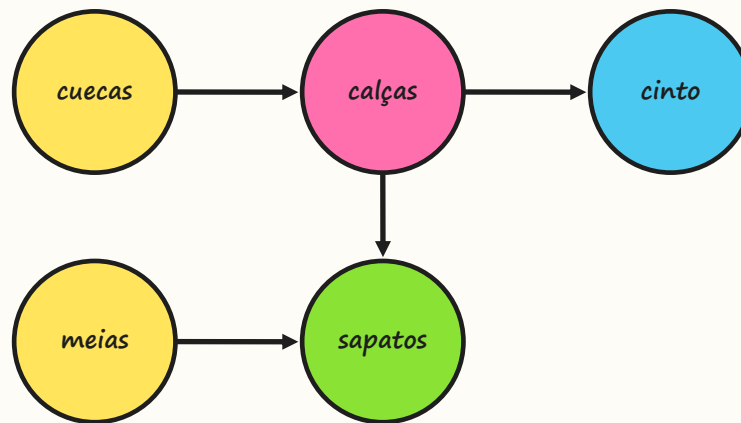
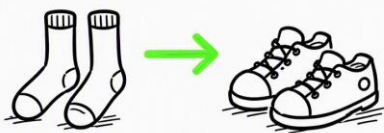
C++ | Flood Fill em Grelha 2D

```
int R, C;
std::vector<std::vector<char>> grelha;
const int dr[] = { -1, 0, 1, 0 };
const int dc[] = { 0, 1, 0, -1 };

int floodFill(int r, int c, char corOriginal, char novaCor) {
    if (r < 0 || r >= R || c < 0 || c >= C) return 0;
    if (grelha[r][c] != corOriginal) return 0;
    grelha[r][c] = novaCor;
    int tamanhoComponente = 1;
    for (int d = 0; d < 4; ++d) {
        tamanhoComponente += floodFill(r + dr[d], c + dc[d], corOriginal, novaCor);
    }
    return tamanhoComponente;
}
```

Ordenação topológica: meias antes dos sapatos

4 · DAG



ordem válida: cuecas → meias → calças → cinto
→ sapatos

Várias ordens podem ser válidas!

Num DAG, ordem linear onde para toda aresta (u, v) , u aparece antes de v . Existe ordenação $\Leftrightarrow$ o grafo é acíclico.

Método	Controlo	Estratégia	Deteção de ciclos
DFS	Pilha de recursão	Adiciona o nó ao concluir e inverte a lista	Requer monitorizar arestas de retorno
Kahn	Fila FIFO ou de prioridade	Remove sucessivamente nós com grau de entrada 0	Ciclo se a ordem final tiver menos de V nós

Topológica na prática: DFS vs Kahn

4 · DAG

DFS: um vértice só termina quando todos os caminhos a partir dele se esgotam → push_back ao sair + reverse.

Kahn: calcula graus de entrada, enfileira os de grau 0, "consome" e decrementa vizinhos. Com fila de prioridade → menor ordem lexicográfica.

C++ | Ordenação Topológica por DFS

```
vi ordemTopologica;

void toposortDFS(int u) {
    dfs_num[u] = VISITADO;
    for (const auto &[v, w] : AL[u]) {
        if (dfs_num[v] == NAO_VISITADO) {
            toposortDFS(v);
        }
    }
    ordemTopologica.push_back(u);
}

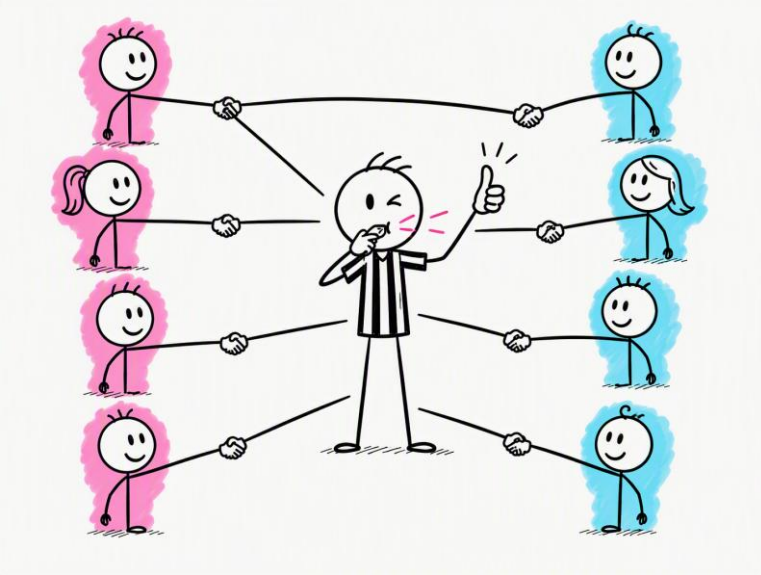
std::vector<int> executarToposort(int V) {
    dfs_num.assign(V, NAO_VISITADO);
    ordemTopologica.clear();
    for (int u = 0; u < V; ++u) {
        if (dfs_num[u] == NAO_VISITADO) {
            toposortDFS(u);
        }
    }
    std::reverse(ordemTopologica.begin(), ordemTopologica.end());
    return ordemTopologica;
}
```

C++ | Algoritmo de Kahn

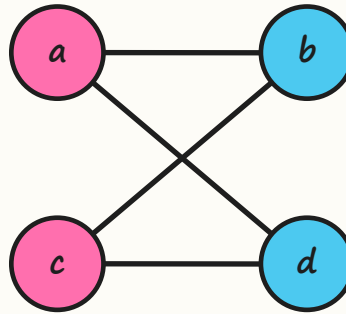
```
std::vector<int> kahnToposort(int V, const std::vector<vii> &AL) {
    vi grauEntrada(V, 0);
    for (int u = 0; u < V; ++u)
        for (const auto &[v, w] : AL[u]) grauEntrada[v]++;
    std::queue<int> fila;
    for (int u = 0; u < V; ++u)
        if (grauEntrada[u] == 0) fila.push(u);
    std::vector<int> resultado;
    while (!fila.empty()) {
        int u = fila.front();
        fila.pop();
        resultado.push_back(u);
        for (const auto &[v, w] : AL[u]) {
            if (--grauEntrada[v] == 0) {
                fila.push(v);
            }
        }
    }
    return resultado;
}
```

Grafo bipartido = 2 cores bastam

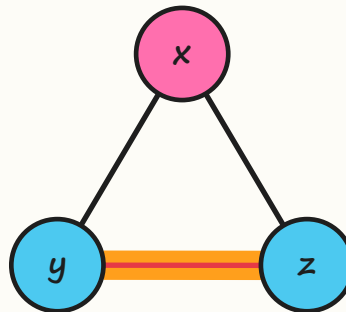
5 · BIPARTIÇÃO



V divide-se em V_1 e V_2 e toda aresta liga V_1 a V_2 . König: bipartido $\Leftrightarrow$ não há ciclo de comprimento ímpar. Colore com BFS/DFS: vizinho com a mesma cor $\rightarrow$ falha!



ciclo par ✓



ciclo ímpar X conflito!

C++ | Verificação de Grafo Bipartido

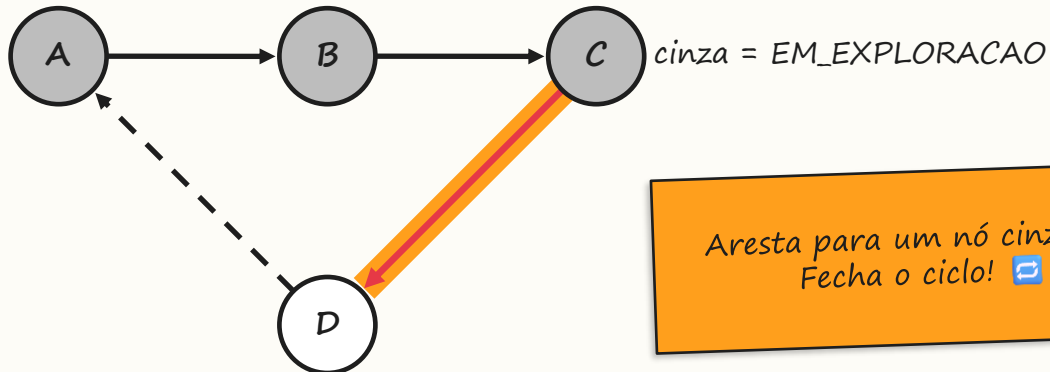
```
bool verificarBipartido(int s, int V, const std::vector<vii> &AL) {
    vi cor(V, -1);
    std::queue<int> q;
    cor[s] = 0;
    q.push(s);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (const auto &[v, w] : AL[u]) {
            if (cor[v] == -1) {
                cor[v] = 1 - cor[u];
                q.push(v);
            } else if (cor[v] == cor[u]) {
                return false;
            }
        }
    }
    return true;
}
```

Ciclos em grafos dirigidos

5 · CICLOS

A DFS classifica as arestas pelo estado do destino: NAO_VISITADO, EM_EXPLORACAO, PROCESSADO.

Aresta	Estado do destino	Significado
Árvore (tree)	NAO_VISITADO	Descobre um novo vértice
Retorno (back)	EM_EXPLORACAO	Liga a um antecessor ativo: CICLO!
Avanço (forward)	PROCESSADO	Liga a um descendente já concluído
Cruzada (cross)	PROCESSADO	Liga ramos distintos, sem ascendência



Aresta para um nó cinzento?
Fecha o ciclo! 🔄

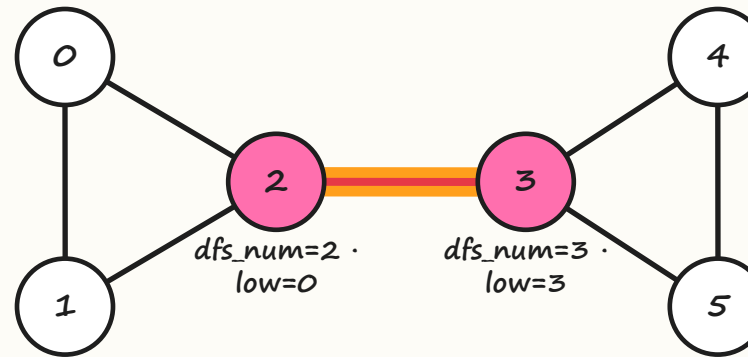
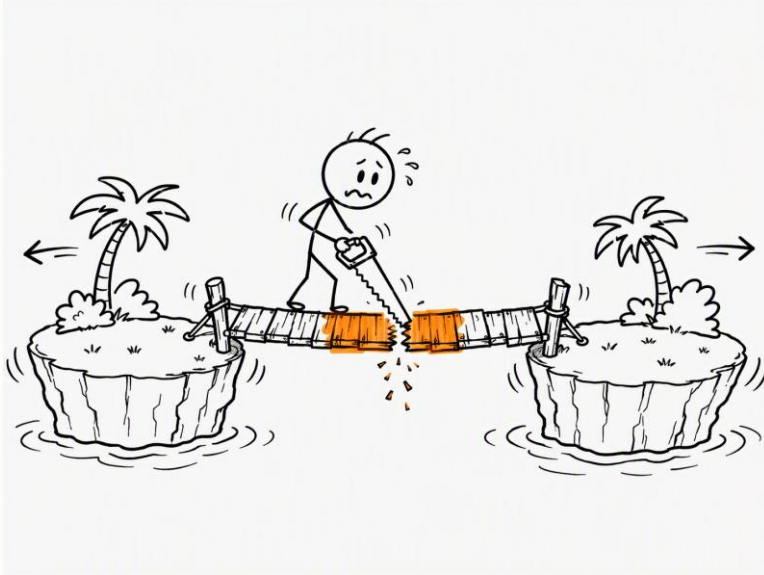
C++ | Detecção de Ciclos em Grafos Dirigidos

```
enum { NAO_VISITADO = -1, EM_EXPLORACAO = -2, PROCESSADO = -3 };
std::vector<vii> AL;
vi dfs_num;
vi dfs_parent;
bool contemCiclo = false;

void cycleCheck(int u) {
    dfs_num[u] = EM_EXPLORACAO;
    for (const auto &[v, w] : AL[u]) {
        if (dfs_num[v] == NAO_VISITADO) {
            dfs_parent[v] = u;
            cycleCheck(v);
        } else if (dfs_num[v] == EM_EXPLORACAO) {
            contemCiclo = true;
        }
    }
    dfs_num[u] = PROCESSADO;
}
```

Pontos de articulação e pontes

6 · CORTES



rosa = articulação · laranja = ponte (2-3)

Sem aresta de retorno por cima de u?
Então u é o único caminho! ✂

Tarjan numa única DFS, $O(V + E)$

- $dfs_num[u]$: ordem de descoberta
- $dfs_low[u]$: menor dfs_num alcançável pela subárvore de u usando no máximo 1 aresta de retorno
- Raiz: articulação $\Leftrightarrow \geq 2$ filhos na árvore DFS
- Não-raiz u: articulação se algum filho v tem $dfs_low[v] \geq dfs_num[u]$
- **Ponte (u, v) $\Leftrightarrow dfs_low[v] > dfs_num[u]$**

- ▶ **Ponto de articulação:** vértice cuja remoção desconecta o grafo.
- ▶ **Ponte:** aresta cuja remoção desconecta o grafo.
- ▶ Revelam gargalos e pontos únicos de falha em redes.

Tarjan para cortes: o código

6 · CORTES

C++ | Pontos de Articulação e Pontes (Tarjan)

```
std::vector<vii> AL;
vi dfs_num, dfs_low, dfs_parent, articulacao;
int dfsContador, dfsRaiz, filhosRaiz;

void encontrarCortes(int u) {
    dfs_low[u] = dfs_num[u] = dfsContador++;
    for (const auto &[v, w] : AL[u]) {
        if (dfs_num[v] == -1) {
            dfs_parent[v] = u;
            if (u == dfsRaiz) ++filhosRaiz;
            encontrarCortes(v);
            if (dfs_low[v] >= dfs_num[u]) {
                articulacao[u] = 1;
            }
            if (dfs_low[v] > dfs_num[u]) {
                // Aresta (u, v) constitui uma ponte
            }
            dfs_low[u] = std::min(dfs_low[u], dfs_low[v]);
        } else if (v != dfs_parent[u]) {
            dfs_low[u] = std::min(dfs_low[u], dfs_num[v]);
        }
    }
}
```

$\geq \rightarrow$ ponto de articulação

$> \rightarrow$ ponte

Aresta de retorno ($v \neq \text{pai}$): puxa $\text{dfs_low}[u]$ para cima

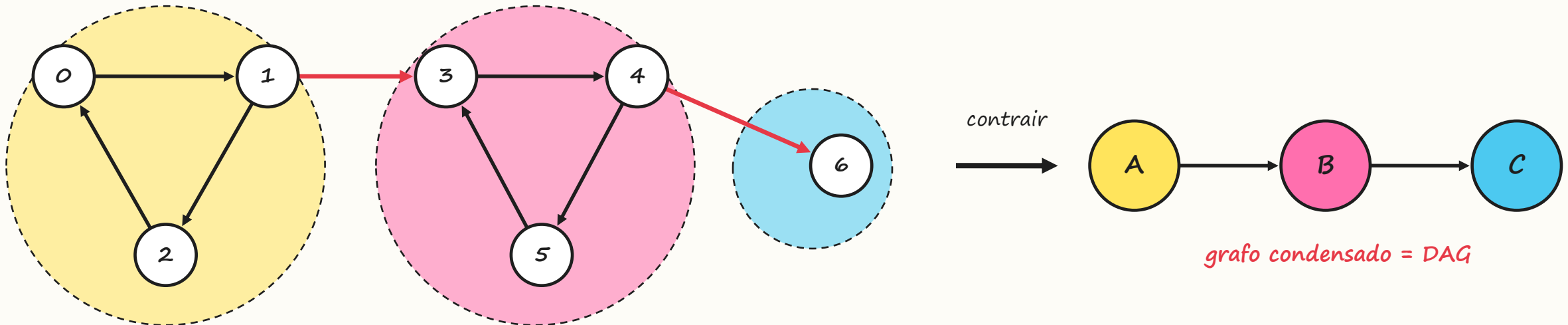
Atenção: a raiz é caso especial — articulação só se $\text{filhosRaiz} \geq 2$

Complexidade: $O(V + E)$ — uma única travessia!

Componentes fortemente conexos (SCC)

7 · SCC

SCC: subgrafo maximal onde, para todo par (u, v) , existe caminho $u \rightarrow v$ e $v \rightarrow u$. Contraindo cada SCC num super-vértice obtém-se o grafo condensado — sempre um DAG, o que permite aplicar Programação Dinâmica.



Tarjan para SCC, $O(V + E)$

- DFS + pilha explícita com os vértices em análise.
- $\text{dfs_low}[u]$ minimizado por arestas de retorno e cruzadas cujo destino ainda está na pilha.
- Se $\text{dfs_low}[u] == \text{dfs_num}[u]$, u é a raiz da SCC: desempilha tudo até u .

Tarjan SCC: o código

7 · SCC

C++ | Componentes Fortemente Conexas (Tarjan)

```
std::vector<vii> AL;
vi dfs_num, dfs_low, noNaPilha;
std::stack<int> pilha;
int contadorTempo = 0, totalSCC = 0;

void tarjanSCC(int u) {
    dfs_low[u] = dfs_num[u] = contadorTempo++;
    pilha.push(u);
    noNaPilha[u] = 1;
    for (const auto &[v, w] : AL[u]) {
        if (dfs_num[v] == -1) {
            tarjanSCC(v);
            dfs_low[u] = std::min(dfs_low[u], dfs_low[v]);
        } else if (noNaPilha[v]) {
            dfs_low[u] = std::min(dfs_low[u], dfs_num[v]);
        }
    }
    if (dfs_low[u] == dfs_num[u]) {
        ++totalSCC;
        while (true) {
            int v = pilha.top();
            pilha.pop();
            noNaPilha[v] = 0;
            if (u == v) break;
        }
    }
}
```

pilha

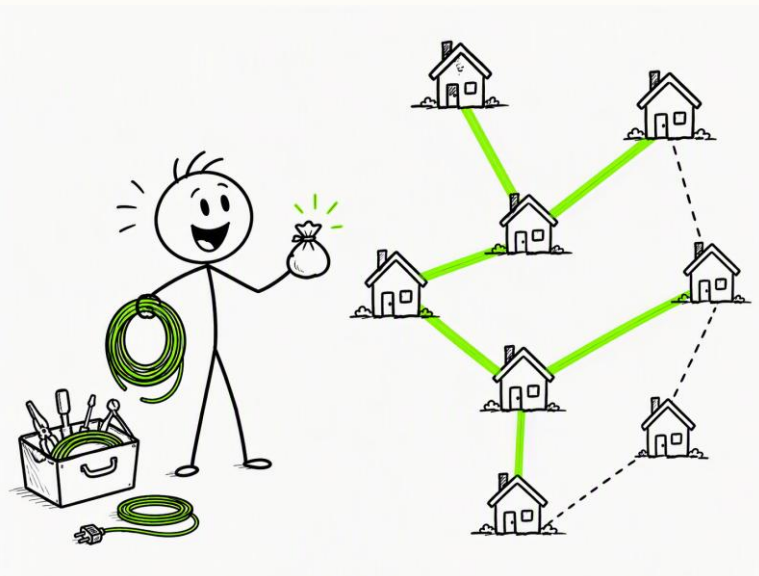


*low == num em 3 →
sai {5,4,3}*

*Cada desempilhamento completo = 1 SCC nova
(++totalSCC)*

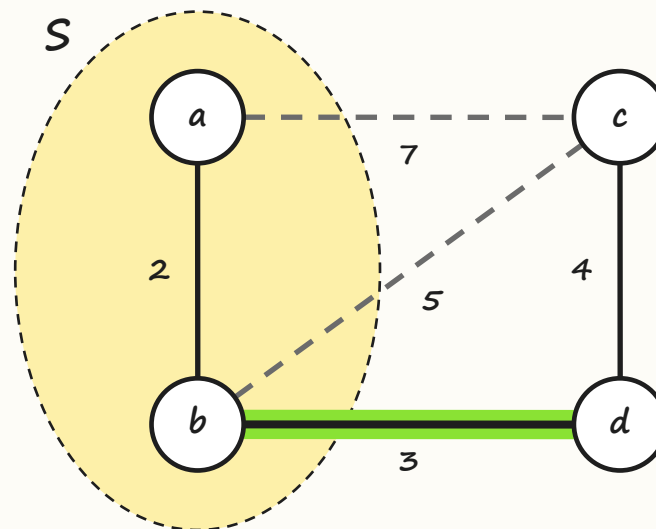
Árvore de cobertura mínima (MST)

8 · MST



Subconjunto $T \subseteq E$ que liga todos os $|V|$ vértices, sem ciclos, com soma de pesos $\sum w(e)$ mínima.

Teorema do Corte: a aresta mais leve que cruza qualquer corte $(S, V \setminus S)$ pertence a uma MST ótima.



corte: a mais leve que cruza (3) entra!

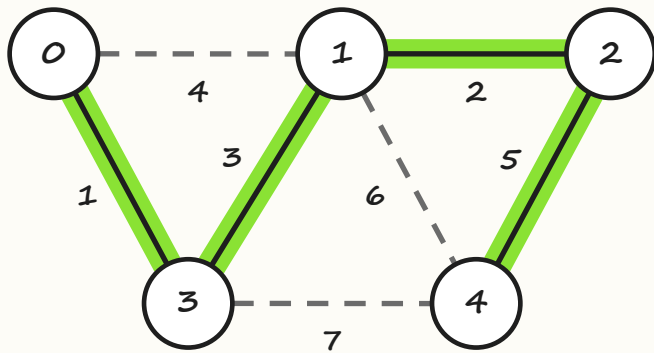
Ligar tudo gastando o mínimo de cabo ⚡

Algoritmo	Suporte	Tempo	Comportamento
Kruskal	Union-Find (UFDS)	$O(E \log E)$	Junta componentes isoladas, arestas ordenadas globalmente
Prim	Fila de prioridade mínima	$O(E \log V)$	Faz crescer uma única componente a partir da raiz

Kruskal: aresta mais barata que não fecha ciclo

8 · MST

1) ordena E por peso · 2) aceita a aresta se u e v estão em conjuntos diferentes (UFDS com union by rank + path compression $\approx \alpha(V)$).



$w=1$ (0,3) ✓
 $w=2$ (1,2) ✓
 $w=3$ (1,3) ✓
 $w=4$ (0,1) ✗ ciclo!
 $w=5$ (2,4) ✓ → V-1
arestas, fim

$\text{custo} = 1 + 2 + 3 + 5 = 11$

$\text{findSet}(u) == \text{findSet}(v)$? Já estão ligados → descarta!

C++ | Algoritmo de Kruskal com Union-Find

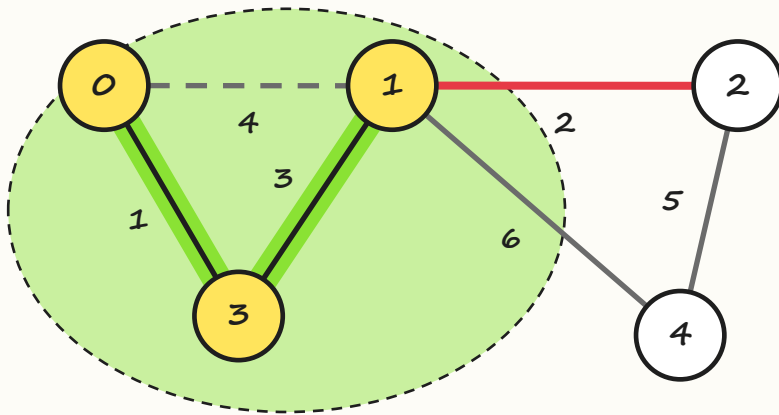
```
class UnionFind {
private:
    vi p, rank;
public:
    UnionFind(int N) {
        p.assign(N, 0); for (int i = 0; i < N; ++i) p[i] = i;
        rank.assign(N, 0);
    }
    int findSet(int i) { return (p[i] == i) ? i : (p[i] = findSet(p[i])); }
    bool isSameSet(int i, int j) { return findSet(i) == findSet(j); }
    void unionSet(int i, int j) {
        if (!isSameSet(i, j)) {
            int x = findSet(i), y = findSet(j);
            if (rank[x] > rank[y]) std::swap(x, y);
            p[x] = y;
            if (rank[x] == rank[y]) ++rank[y];
        }
    }
};

int kruskalMST(int V, std::vector<iii> &EL) {
    std::sort(EL.begin(), EL.end());
    UnionFind UF(V);
    int custoMST = 0, arestasContabilizadas = 0;
    for (const auto &[w, u, v] : EL) {
        if (!UF.isSameSet(u, v)) {
            custoMST += w;
            UF.unionSet(u, v);
            if (++arestasContabilizadas == V - 1) break;
        }
    }
    return custoMST;
}
```

Prim: a árvore cresce como uma mancha

8 · MST

Parte de um nó; a fila de prioridade mínima guarda as arestas que atravessam o corte (árvore | resto). Extrai a mais leve que chega a um vértice novo.



$pq = \{(2, \rightarrow 2), (4, \rightarrow 0 \text{ X tomado}), (6, \rightarrow 4)\} \rightarrow \text{tira } (2)!$

if (tomado[u]) continue; — entradas velhas são só ignoradas

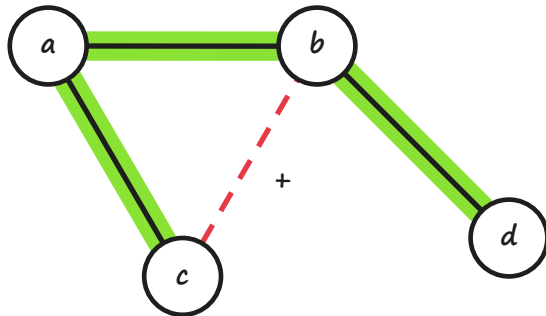
C++ | Algoritmo de Prim

```
std::vector<vii> AL;  
vi tomado;  
std::priority_queue<ii, std::vector<ii>, std::greater<ii>> pq;  
  
void processarVertice(int u) {  
    tomado[u] = 1;  
    for (const auto &[v, w] : AL[u]) {  
        if (!tomado[v]) {  
            pq.push({w, v});  
        }  
    }  
}  
  
int primMST(int V) {  
    tomado.assign(V, 0);  
    processarVertice(0);  
    int custoMST = 0, arestasContabilizadas = 0;  
    while (!pq.empty()) {  
        auto [w, u] = pq.top();  
        pq.pop();  
        if (tomado[u]) continue;  
        custoMST += w;  
        processarVertice(u);  
        if (++arestasContabilizadas == V - 1) break;  
    }  
    return custoMST;  
}
```

MST em modo avançado

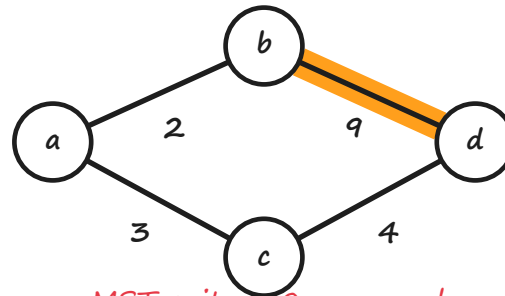
8 · MST

Segunda melhor MST



Constrói a MST T . Cada aresta $(u,v) \notin T$ cria exatamente 1 ciclo: troca-a pela aresta mais pesada desse ciclo. Escolhe a troca de menor acréscimo. Máximo no caminho $u-v$ em $O(\log V)$ com LCA + binary lifting.

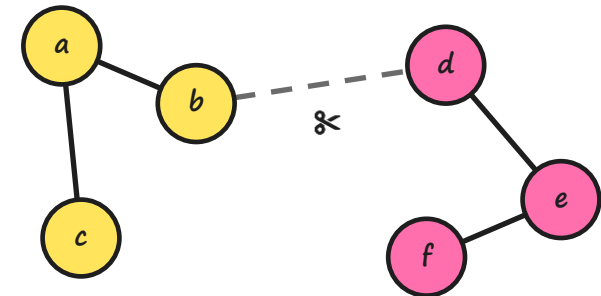
Gargalo mínimo



a MST evita o 9 $\rightarrow$ gargalo = 4

Minimizar o peso da aresta mais pesada da cobertura. Pelo Teorema do Corte, qualquer MST (Kruskal ou Prim) já é uma árvore de gargalo mínimo — sem modificações!

Clustering em K grupos



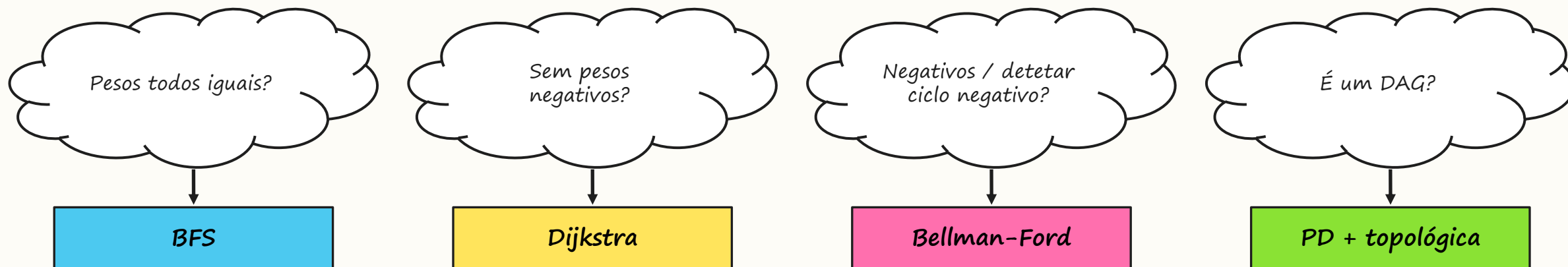
Kruskal com paragem antecipada: começa com $|V|$ componentes, cada junção reduz 1. Para ao atingir K . O peso da próxima aresta = separação máxima entre grupos.

Caminhos mínimos de fonte única (SSSP)

9 · CAMINHOS

Menor soma de pesos de uma origem fixa s a todos os $v \in V$. Os pesos e a existência de ciclos decidem o algoritmo:

Algoritmo	Pesos	Topologia	Tempo	Princípio
BFS	Uniformes / unitários	Qualquer grafo	$O(V + E)$	Travessia concêntrica por níveis
Dijkstra	Não-negativos ($w \geq 0$)	Qualquer grafo	$O((V + E) \log V)$	Relaxamento guloso com fila de prioridade
Bellman-Ford	Arbitrários (aceita negativos)	Sem ciclos negativos	$O(V \cdot E)$	Rondas de relaxamento em todas as arestas
PD em DAG	Arbitrários	Grafos acíclicos dirigidos	$O(V + E)$	Relaxa seguindo a ordem topológica

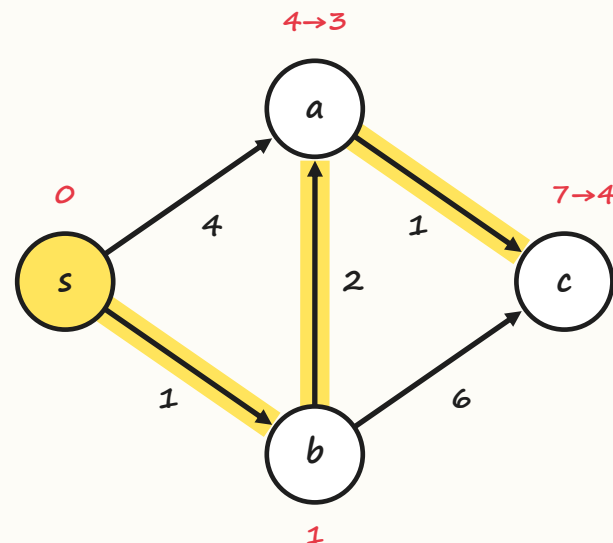


Dijkstra: sempre o mais perto primeiro

9 · CAMINHOS



Pesos $\geq 0 \Rightarrow$ distâncias não decrescem ao longo do caminho. Extrai o u de menor $dist$ provisória; entradas com $d > dist[u]$ são descartadas. Relaxa: $dist[u] + w < dist[v]$.



Relaxar = "achei atalho? atualiza!"

ordem de extração: $s(0) \rightarrow b(1) \rightarrow a(3) \rightarrow c(4)$

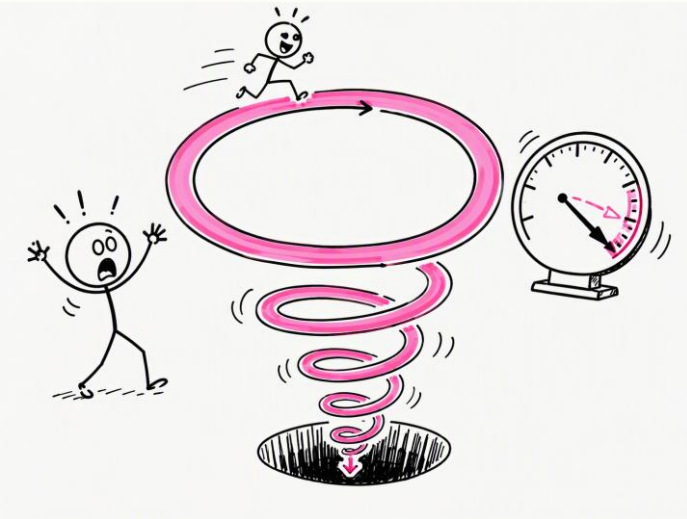
C++ | Algoritmo de Dijkstra

```
const int INF = 1e9;
std::vector<vii> AL;
vi dist;

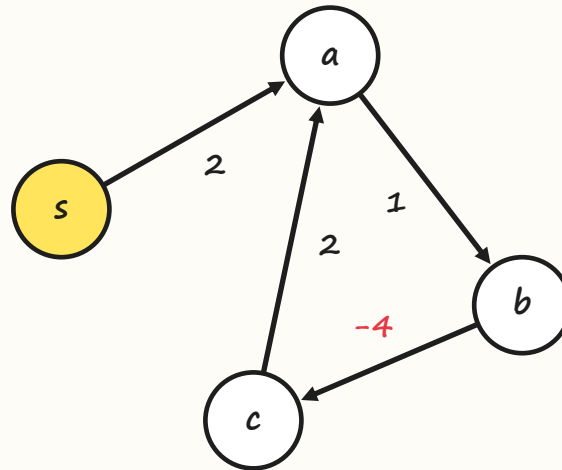
void dijkstra(int s, int V) {
    dist.assign(V, INF);
    dist[s] = 0;
    std::priority_queue<ii, std::vector<ii>, std::greater<ii>> pq;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (d > dist[u]) continue;
        for (const auto &[v, w] : AL[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
}
```

Bellman-Ford: paciente e desconfiado

9 · CAMINHOS



- ▶ Pesos negativos quebram o guloso do Dijkstra.
- ▶ **|V| - 1 rondas** de relaxamento sobre todas as arestas: um caminho simples mínimo tem $\leq |V| - 1$ arestas.
- ▶ **V-ésima ronda:** se ainda relaxa $\rightarrow$ ciclo de peso negativo (distância $\rightarrow -\infty$).



ciclo $a \rightarrow b \rightarrow c \rightarrow a = 1 - 4 + 2 = -1$ 🤖

*Cada volta fica mais barata...
para sempre!*

C++ | Algoritmo de Bellman-Ford

```
const int INF = 1e9;
vi dist;

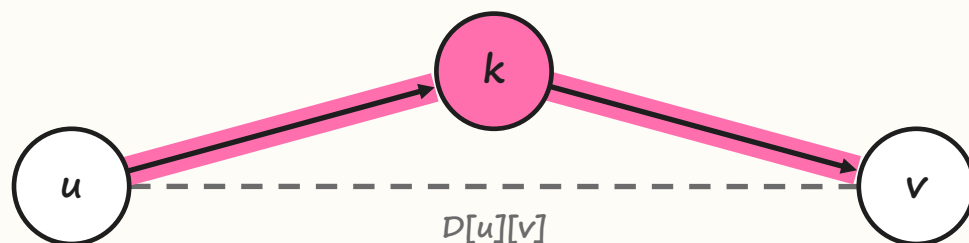
bool bellmanFord(int s, int V, const std::vector<iii> &arestas) {
    dist.assign(V, INF);
    dist[s] = 0;
    for (int i = 0; i < V - 1; ++i) {
        for (const auto &[u, v, w] : arestas) {
            if (dist[u] != INF && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
            }
        }
    }
    for (const auto &[u, v, w] : arestas) {
        if (dist[u] != INF && dist[u] + w < dist[v]) {
            return false;
        }
    }
    return true;
}
```

Floyd-Warshall: todos os pares (APSP)

9 · CAMINHOS

Programação Dinâmica: o melhor caminho $u \rightarrow v$ usando apenas intermediários $\{0, 1, \dots, k\}$:

$$D^{(k)}[u][v] = \min(D^{(k-1)}[u][v], D^{(k-1)}[u][k] + D^{(k-1)}[k][v])$$



passar por k compensa?

- ▶ Tabela 3D projetada na própria matriz de adjacência 2D.
- ▶ $O(V^3)$ com três laços: k TEM de ser o laço mais externo!
- ▶ **Fecho transitivo:** $AM[u][v] \mid= (AM[u][k] \ \& \ AM[k][v])$.
- ▶ **Ciclo negativo:** $AM[i][i] < 0$ na diagonal no fim.

C++ | Algoritmo de Floyd-Warshall

```
const int INF = 1e9;
const int MAX_V = 450;
int AM[MAX_V][MAX_V];

void floydWarshall(int V) {
    for (int k = 0; k < V; ++k) {
        for (int u = 0; u < V; ++u) {
            for (int v = 0; v < V; ++v) {
                if (AM[u][k] < INF && AM[k][v] < INF) {
                    AM[u][v] = std::min(AM[u][v], AM[u][k] + AM[k][v]);
                }
            }
        }
    }
}
```

$V \leq 450 \rightarrow 450^3 \approx 9 \times 10^7$ operações ✓

Grafos especiais: atalhos de ouro

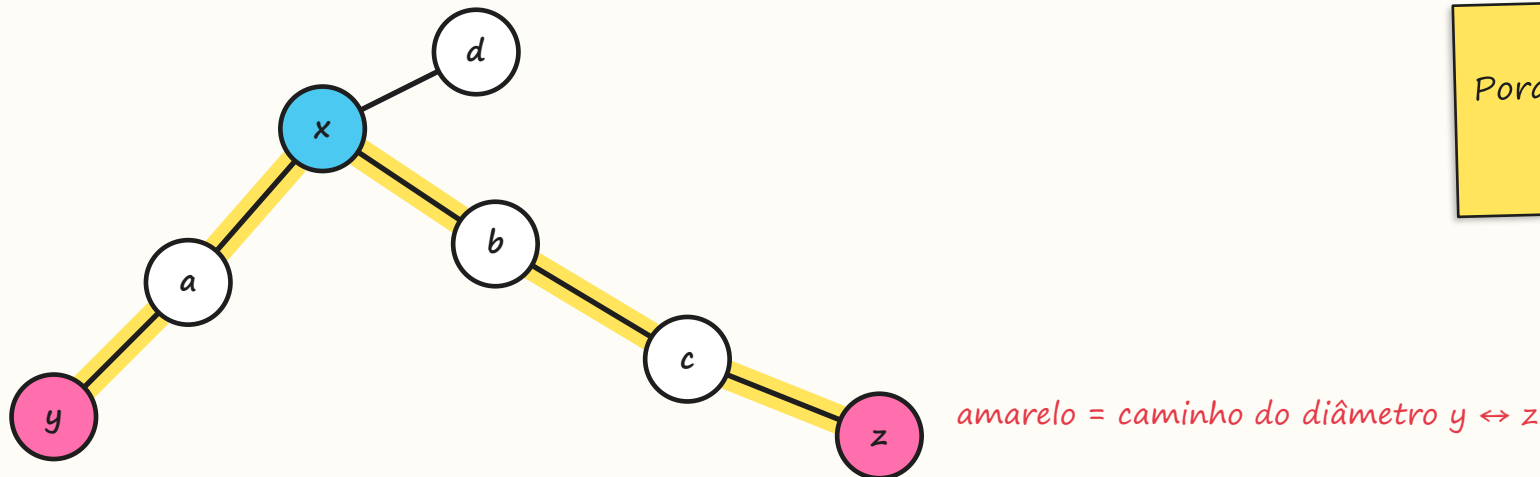
10 · ESPECIAIS

Certas topologias admitem soluções muito mais baratas que as formulações gerais.

Família	Propriedades	Aplicação algorítmica
Árvore	Conexa, acíclica, $ E = V - 1$	Diâmetro em $O(V)$ com duas travessias consecutivas
DAG	Sem ciclos orientados; admite ordenação linear	Caminhos mínimos e máximos em $O(V + E)$
Grafo Euleriano	Percurso fechado/aberto que usa cada aresta 1 vez	Circuito em $O(V + E)$ por Hierholzer

- 1ª busca: de x qualquer $\rightarrow$ acha y , o mais distante
- 2ª busca: de $y \rightarrow$ acha z , o mais distante de y
3. **$\text{dist}(y, z) = \text{diâmetro}$** 🌀

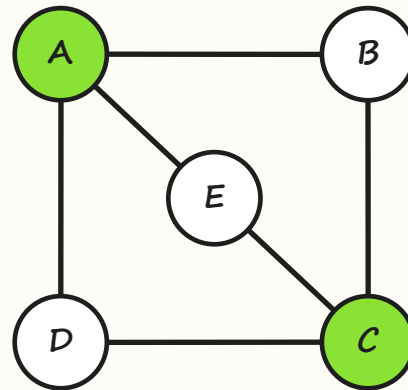
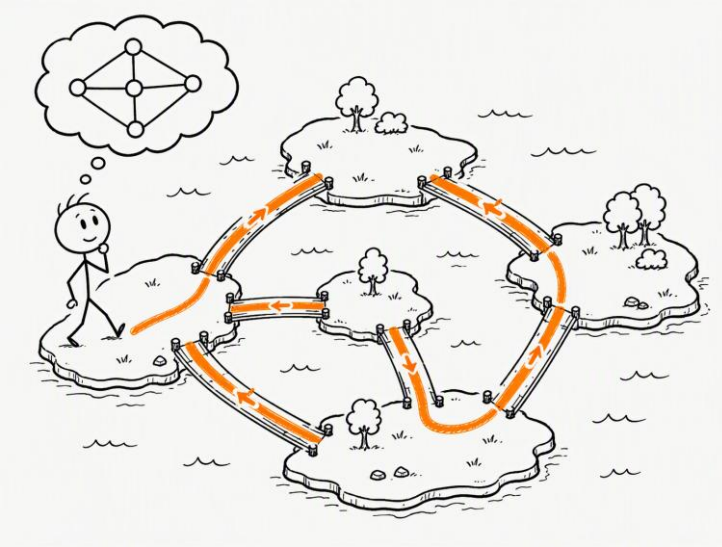
Diâmetro de uma árvore (pesos ≥ 0) com 2 BFS/DFS:



Porquê funciona? O nó mais distante de qualquer x é sempre uma ponta de um diâmetro.

Euler: cada aresta exatamente uma vez

10 · EULER



graus: A=3, C=3 (ímpares) · B=D=E=2

Regras de paridade

- Não dirigido e conexo: circuito $\Leftrightarrow$ todos os graus pares.
- Exatamente 2 graus ímpares $\Rightarrow$ caminho euleriano aberto.
- Dirigido (fracamente conexo): circuito $\Leftrightarrow$ grauEntrada(u) = grauSaida(u) para todo u.

2 ímpares $\rightarrow$ caminho aberto de A até C ✓

Hierholzer: pilha avança por arestas livres; nó sem saídas $\rightarrow$ vai para a resposta. No fim, inverte.

C++ | Algoritmo de Hierholzer

```
std::vector<std::vector<int>>> AL;

std::vector<int> hierholzer(int s, int V) {
    std::vector<int> resultado, pilha, idx(V, 0);
    pilha.push_back(s);
    while (!pilha.empty()) {
        int u = pilha.back();
        if (idx[u] < (int)AL[u].size()) {
            int v = AL[u][idx[u]++];
            pilha.push_back(v);
        } else {
            resultado.push_back(u);
            pilha.pop_back();
        }
    }
    std::reverse(resultado.begin(), resultado.end());
    return resultado;
}
```

Quadro de decisão em prova

11 · DECISÃO

Escolha pela análise conjunta de tempo, memória e estrutura do grafo no enunciado:

Objetivo	Restrições típicas	Estrutura de suporte	Tempo	Algoritmo canônico
Exploração e conectividade	$V, E \leq 2 \times 10^5$	Vetor de marcação	$O(V + E)$	DFS
Caminho mínimo não ponderado	$V, E \leq 2 \times 10^5$	Fila FIFO	$O(V + E)$	BFS
Grelhas matriciais 2D	$R \times C \leq 10^6$	Recursão de grelha	$O(R \cdot C)$	Flood Fill
Dependências em grafo dirigido	$V, E \leq 2 \times 10^5$	Vetor de graus de entrada	$O(V + E)$	Kahn ou DFS com reversão
Bipartição / 2-colorabilidade	$V, E \leq 2 \times 10^5$	Vetor de duas cores	$O(V + E)$	Bicoloração BFS/DFS
Articulações e pontes	$V, E \leq 10^5$	dfs_num e dfs_low	$O(V + E)$	Tarjan (cortes)
Componentes fortemente conexas	$V, E \leq 10^5$	Pilha explícita e dfs_low	$O(V + E)$	Tarjan (SCC)
Árvore de cobertura mínima	$E \leq 3 \times 10^5$	Conjuntos disjuntos (UFDS)	$O(E \log E)$	Kruskal
Caminho mínimo ($w \geq 0$)	$V, E \leq 2 \times 10^5$	Fila de prioridade mínima	$O((V + E) \log V)$	Dijkstra
Caminho mínimo com ciclos negativos	$V \leq 2500, E \leq 10^4$	Lista de arestas	$O(V \cdot E)$	Bellman-Ford
Caminhos entre todos os pares	$V \leq 450$	Matriz de adjacência	$O(V^3)$	Floyd-Warshall
Percurso de arestas sem repetição	$V, E \leq 2 \times 10^5$	Pilha e índices de aresta	$O(V + E)$	Hierholzer

Agora é praticar!

1. Modele o problema como grafo.
2. Leia as restrições de V e E.
3. Escolha a estrutura e o algoritmo.
4. **Implemente, teste, submeta... Accepted!** ✓

